

REGRA DE HEBB APLICADA ÀS FUNÇÕES LÓGICAS DE 2

1 Objetivo

Este material tem um objetivo simples: mostrar, na prática, até onde vai a capacidade de aprendizado de um único neurônio treinado pela **regra de Hebb**.

Vamos usar as 16 funções lógicas possíveis com 2 variáveis binárias (AND, OR, XOR, etc.) como exemplos de treinamento. A pergunta é: **quantas dessas funções um neurônio simples consegue aprender usando apenas a regra de Hebb?**

A resposta, que vamos demonstrar matematicamente e depois confirmar em Python, é: **14 das 16**. As duas exceções são a função XOR e sua função complementar, XNOR.

2 O neurônio e a representação bipolar

Usaremos um neurônio bem simples, com duas entradas x_1 e x_2 , dois pesos w_1 e w_2 , um bias b , e uma saída y dada por:

$$y = \text{sinal}(w_1x_1 + w_2x_2 + b)$$

A função sinal (ou *sign*) devolve +1 se o valor dentro dela for ≥ 0 , e -1 caso contrário.

Representação bipolar: em vez de usar 0 e 1 como nos costumamos ver em lógica booleana, vamos representar falso como -1 e verdadeiro como +1:

$$0 \rightarrow -1 \qquad 1 \rightarrow +1$$

Essa mudança não é apenas cosmética – é o que faz a regra de Hebb funcionar bem, como veremos.

3 A regra de Hebb

A ideia da regra de Hebb, proposta em 1949 [1], é simples: *reforçar a conexão entre entrada e saída sempre que elas “concordam”*. Matematicamente, para cada exemplo de treinamento (x_1, x_2, t) , onde t é a saída desejada (alvo):

$$w_1 += x_1 \cdot t \qquad w_2 += x_2 \cdot t \qquad b += t$$

Começando com $w_1 = w_2 = b = 0$ e passando pelos 4 exemplos de treinamento uma única vez (sem repetir, sem iterar), os pesos finais são simplesmente a soma acumulada:

$$w_1 = \sum_{p=1}^4 x_1^{(p)} t^{(p)} \qquad w_2 = \sum_{p=1}^4 x_2^{(p)} t^{(p)} \qquad b = \sum_{p=1}^4 t^{(p)}$$

É só isso – não há iteração, nem taxa de aprendizado, nem gradiente. É o algoritmo de treinamento mais simples que existe.

Exemplo: treinando a função AND

A tabela-verdade da função AND, em bipolar, é:

x_1	x_2	t (alvo)
-1	-1	-1
-1	+1	-1
+1	-1	-1
+1	+1	+1

Aplicando a fórmula:

$$w_1 = (-1)(-1) + (-1)(-1) + (1)(-1) + (1)(1) = 1 + 1 - 1 + 1 = 2$$

$$w_2 = (-1)(-1) + (1)(-1) + (-1)(-1) + (1)(1) = 1 - 1 + 1 + 1 = 2$$

$$b = (-1) + (-1) + (-1) + (1) = -2$$

O neurônio treinado fica $y = \text{sin}(2x_1 + 2x_2 - 2)$. Conferindo nos 4 pontos:

(x_1, x_2)	$2x_1 + 2x_2 - 2$	saída
$(-1, -1)$	-6	-1 ✓
$(-1, +1)$	-2	-1 ✓
$(+1, -1)$	-2	-1 ✓
$(+1, +1)$	+2	+1 ✓

Funcionou perfeitamente! Todos os 4 pontos foram classificados como deveriam.

4 As 16 funções e o resultado geral

Existem $2^4 = 16$ funções lógicas possíveis com 2 entradas binárias (cada uma das 4 combinações de entrada pode gerar saída 0 ou 1). Repetindo o mesmo procedimento do exemplo do AND para todas elas, obtemos:

Tabela 1: Regra de Hebb aplicada às 16 funções lógicas de 2 variáveis.

Função	Alvo ($t^{(1)}, \dots, t^{(4)}$)	w_1	w_2	b	Aprendeu?
FALSO (0)	$(-1, -1, -1, -1)$	+0	+0	-4	✓
AND	$(-1, -1, -1, +1)$	+2	+2	-2	✓
x_1 AND NOT x_2	$(-1, -1, +1, -1)$	+2	-2	-2	✓
x_1	$(-1, -1, +1, +1)$	+4	+0	+0	✓
NOT x_1 AND x_2	$(-1, +1, -1, -1)$	-2	+2	-2	✓
x_2	$(-1, +1, -1, +1)$	+0	+4	+0	✓
XOR	$(-1, +1, +1, -1)$	0	0	0	×
OR	$(-1, +1, +1, +1)$	+2	+2	+2	✓
NOR	$(+1, -1, -1, -1)$	-2	-2	-2	✓
XNOR	$(+1, -1, -1, +1)$	0	0	0	×
NOT x_2	$(+1, -1, +1, -1)$	+0	-4	+0	✓
x_1 OR NOT x_2	$(+1, -1, +1, +1)$	+2	-2	+2	✓
NOT x_1	$(+1, +1, -1, -1)$	-4	+0	+0	✓
NOT x_1 OR x_2	$(+1, +1, -1, +1)$	-2	+2	+2	✓
NAND	$(+1, +1, +1, -1)$	-2	-2	+2	✓
VERDADEIRO (1)	$(+1, +1, +1, +1)$	+0	+0	+4	✓

Resultado: 14 de 16 funções aprendidas com sucesso. Repare que, exatamente nos dois casos que falham (XOR e XNOR), os pesos calculados pela regra de Hebb são todos zero – o neurônio simplesmente não consegue "decidir" nada.

5 Por que XOR não funciona?

A razão é geométrica. O neurônio que estamos usando só consegue separar duas classes com uma **reta** no plano (x_1, x_2) – por isso dizemos que ele resolve apenas funções **linearmente separáveis**.

Olhando os pontos da função XOR:

- classe +1: os pontos $(-1, +1)$ e $(+1, -1)$
- classe -1: os pontos $(-1, -1)$ e $(+1, +1)$

As duas classes ficam em **cantos opostos** do quadrado – é impossível traçar uma única reta que separe esses pontos corretamente. Por isso XOR (e sua versão invertida, XNOR) é o exemplo clássico usado em sala de aula para mostrar os limites de um único neurônio, e foi justamente essa limitação, apontada por Minsky e Papert em 1969 [3], que motivou mais tarde o desenvolvimento das redes com múltiplas camadas.

A Figura 1 mostra a diferença visualmente: à esquerda, AND é separável por uma reta; à direita, XOR não é.

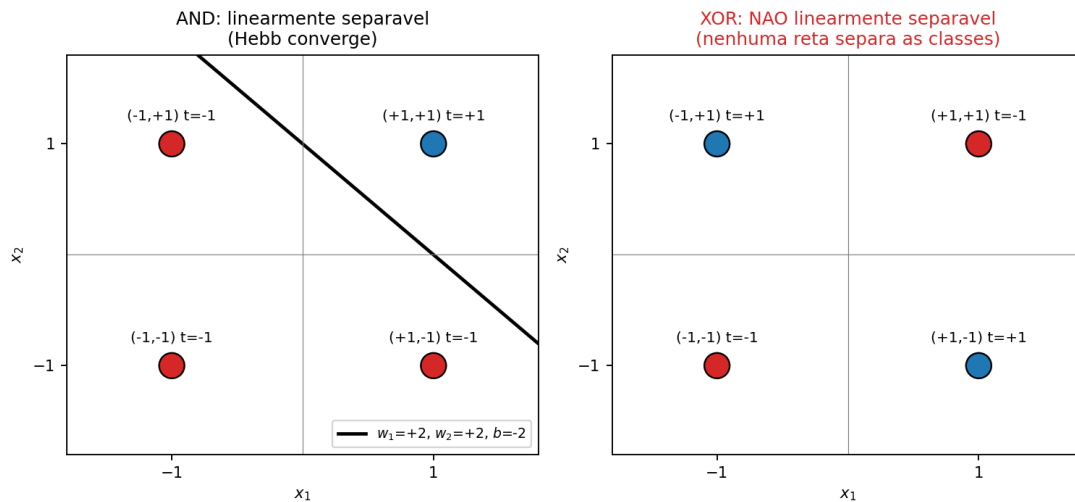


Figura 1: AND (linearmente separável, a regra de Hebb encontra a reta) vs. XOR (não separável, nenhuma reta resolve).

6 Demonstração em Python

O programa `hebb_interativo.py` implementa a regra de Hebb de forma interativa: o usuário escolhe, em um menu, qual das 16 funções lógicas deseja treinar. O programa então mostra o cálculo passo a passo dos pesos, testa a rede nos 4 padrões de entrada e gera o gráfico da fronteira de decisão correspondente.

```

1 def regra_hebb(X, t):
2     """Calcula w1, w2, b pela regra de Hebb (passagem unica)."""
3     w = np.zeros(2)
4     b = 0.0
5     for x, ti in zip(X, t):
6         w += x * ti
7         b += ti
8     return w[0], w[1], b
9
10 def testa_rede(w1, w2, b, X, t):
11     """Retorna (acertos, saida_predita, sucesso_total)."""
12     saida = []
13     acertos = 0
14     for x, ti in zip(X, t):
15         net = w1 * x[0] + w2 * x[1] + b
16         y = sinal_bipolar(net)
17         saida.append(y)
18         if y == ti:
19             acertos += 1
20     return acertos, saida, acertos == len(t)

```

Ao escolher, por exemplo, a função AND no menu, o programa imprime:

```

1 Funcao escolhida: AND

```

2	-----					
3	(x1,x2)	alvo t		calculo de w1, w2 e b (acumulado)		
4	-----					
5	(-1,-1)	-1	->	w1=+1	w2=+1	b=-1
6	(-1,+1)	-1	->	w1=+2	w2=+0	b=-2
7	(+1,-1)	-1	->	w1=+1	w2=+1	b=-3
8	(+1,+1)	+1	->	w1=+2	w2=+2	b=-2
9	-----					
10	Pesos finais (regra de Hebb): w1=+2 w2=+2 b=-2					
11	Neuronio treinado: y = sinal(+2*x1 +2*x2 -2)					
12	RESULTADO: a regra de Hebb ENCONTROU os pesos corretos (4/4 padroes)					
13	-- funcao linearmente separavel.					

e, em seguida, gera e salva automaticamente o gráfico correspondente na pasta **figs/**. Repetindo esse procedimento para as 16 funções, obtemos exatamente o resultado da Tabela 1: **14 sucessos e 2 falhas (XOR e XNOR)**. A Figura 2 reúne o resultado completo, função por função.

Regra de Hebb (bipolar) aplicada as 16 funcoes logicas de 2 variaveis
(azul = alvo +1, vermelho = alvo -1; reta = fronteira de decisao encontrada por Hebb)

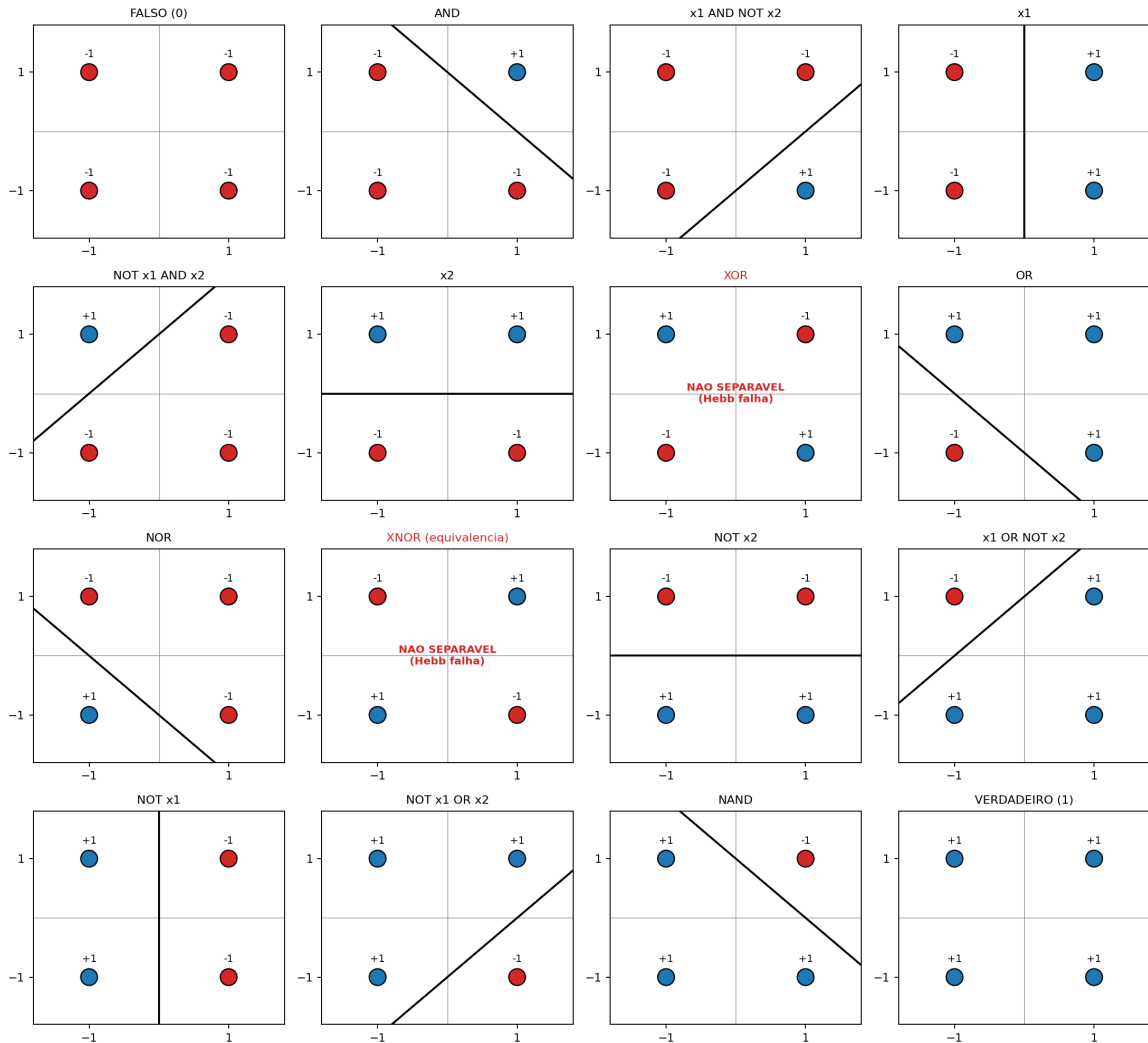


Figura 2: As 16 funções lógicas e as retas de decisão encontradas pela regra de Hebb. Repare que XOR e XNOR (em vermelho) são os únicos casos sem reta separadora.

7 Conclusão

- A regra de Hebb é um algoritmo de treinamento extremamente simples (uma única passagem pelos dados, sem iteração) e, mesmo assim, resolve 14 das 16 funções lógicas de 2 variáveis quando usamos representação bipolar.
- As duas exceções, XOR e XNOR, são as únicas funções que não podem ser separadas por uma reta – ou seja, não são *linearmente separáveis*.
- Esse limite é o mesmo limite de qualquer neurônio único (incluindo o Perceptron), e foi historicamente o que motivou a criação de redes com camadas escondidas.

Referências

- [1] Hebb, D. O. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, New York, 1949.
- [2] Fausett, L. *Fundamentals of Neural Networks: Architectures, Algorithms, and Applications*. Prentice-Hall, Englewood Cliffs, NJ, 1994.
- [3] Minsky, M.; Papert, S. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969 (ed. expandida 1988).

A Código-fonte completo em Python

O código completo do programa interativo usado para gerar a Tabela 1 e as Figuras 1 e 2 deste documento está reproduzido abaixo (hebb_interativo.py).

```
1  """
2  Regra de Hebb - programa interativo
3  -----
4  Escolha uma das 16 funcoes logicas de 2 variaveis em um menu.
5  O programa mostra a tabela-verdade, o calculo passo a passo dos
6  pesos
7  pela regra de Hebb (representacao bipolar) e o teste da rede nos
8  4
9  padroes de entrada. Em seguida, gera e salva o grafico da
10  fronteira
11  de decisao (ou avisa que a funcao nao e linearmente separavel).
12  """
13
14  import os
15  import numpy as np
16  import matplotlib.pyplot as plt
17
18  #
19  -----
20
21  # 1. As 16 funcoes logicas de 2 variaveis (alvo em bipolar)
22  #
23  -----
24
25  # Ordem fixa dos padroes de entrada: (-1,-1), (-1,1), (1,-1),
26  (1,1)
27  X = np.array([[ -1, -1], [-1, 1], [1, -1], [1, 1]], dtype=float)
28
29  FUNCOES = {
30      "1": ("FALSO (0)",          [-1, -1, -1, -1]),
31      "2": ("AND",                [-1, -1, -1, 1]),
32      "3": ("x1 AND NOT x2",      [-1, -1, 1, -1]),
33      "4": ("x1",                 [-1, -1, 1, 1]),
34      "5": ("NOT x1 AND x2",      [-1, 1, -1, -1]),
35      "6": ("x2",                 [-1, 1, -1, 1]),
36      "7": ("XOR",                [-1, 1, 1, -1]),
37      "8": ("OR",                 [-1, 1, 1, 1]),
38      "9": ("NOR",                [1, -1, -1, -1]),
39      "10": ("XNOR (equivalencia)", [1, -1, -1, 1]),
40      "11": ("NOT x2",            [1, -1, 1, -1]),
41      "12": ("x1 OR NOT x2",      [1, -1, 1, 1]),
42      "13": ("NOT x1",            [1, 1, -1, -1]),
43      "14": ("NOT x1 OR x2",      [1, 1, -1, 1]),
44      "15": ("NAND",              [1, 1, 1, -1]),
45      "16": ("VERDADEIRO (1)",    [1, 1, 1, 1]),
46  }
```

```

39
40 PASTA_SAIDA = "figs" # onde os graficos serao salvos
41
42
43 #
44 -----
45
46 # 2. Regra de Hebb
47 #
48 -----
49
50 def regra_hebb(X, t):
51     """Calcula w1, w2, b pela regra de Hebb (passagem unica)."""
52     w = np.zeros(2)
53     b = 0.0
54     for x, ti in zip(X, t):
55         w += x * ti
56         b += ti
57     return w[0], w[1], b
58
59
60 def sinal_bipolar(v):
61     return 1.0 if v >= 0 else -1.0
62
63
64 def testa_rede(w1, w2, b, X, t):
65     """Retorna (acertos, saida_predita, sucesso_total)."""
66     saida = []
67     acertos = 0
68     for x, ti in zip(X, t):
69         net = w1 * x[0] + w2 * x[1] + b
70         y = sinal_bipolar(net)
71         saida.append(y)
72         if y == ti:
73             acertos += 1
74     return acertos, saida, acertos == len(t)
75
76 #
77 -----
78
79 # 3. Exibicao do menu e da tabela-verdade / calculo dos pesos
80 #
81 -----
82
83 def mostra_menu():
84     print("\n" + "=" * 60)
85     print("REGRA DE HEBB - FUNCOES LOGICAS DE 2 VARIAVEIS (")
86         BIPOLAR)")
87     print("=" * 60)

```

```

80     for chave, (nome, _) in FUNCOES.items():
81         print(f"    {chave:>2s}) {nome}")
82     print("    0) Sair")
83     print("=" * 60)
84
85
86 def mostra_resultado(nome, t):
87     t = np.array(t, dtype=float)
88     w1, w2, b = regra_hebb(X, t)
89     acertos, saida, sucesso = testa_rede(w1, w2, b, X, t)
90
91     print(f"\nFuncao escolhida: {nome}")
92     print("-" * 72)
93     print(f"{'(x1,x2)':>10s} {'alvo t':>8s}    |    calculo de w1,
94           w2 e b (acumulado)")
95     print("-" * 72)
96
97     w1_acum = w2_acum = b_acum = 0.0
98     for (x1, x2), ti in zip(X, t):
99         w1_acum += x1 * ti
100        w2_acum += x2 * ti
101        b_acum += ti
102        print(f"({x1:+.0f},{x2:+.0f})    {ti:+.0f}    -> "
103              f"w1={w1_acum:+.0f}  w2={w2_acum:+.0f}  b={b_acum
104                :+.0f}")
105
106     print("-" * 72)
107     print(f"Pesos finais (regra de Hebb): w1={w1:+.0f}  w2={w2
108           :+.0f}  b={b:+.0f}")
109     print(f"Neuronio treinado: y = sinal({w1:+.0f}*x1 {w2:+.0f}*
110           x2 {b:+.0f})")
111     print("-" * 72)
112
113     print(f"{'(x1,x2)':>10s} {'net':>8s} {'saida':>7s} {'alvo
114           ':>6s} {'OK?':>5s}")
115     for (x1, x2), ti, y in zip(X, t, saida):
116         net = w1 * x1 + w2 * x2 + b
117         ok = "SIM" if y == ti else "NAO"
118         print(f"({x1:+.0f},{x2:+.0f})    {net:+6.1f}    {y:+6.0f}
119               {ti:+5.0f}    {ok:>5s}")
120
121     print("-" * 72)
122     if sucesso:
123         print(f"RESULTADO: a regra de Hebb ENCONTROU os pesos
124               corretos "
125               f"({acertos}/4 padroes) -- funcao linearmente
126               separavel.")
127     else:
128         print(f"RESULTADO: a regra de Hebb NAO conseguiu
129               aprender esta funcao ")

```

```

121         f"({acertos}/4 padroes corretos) -- funcao NAO
           linearmente separavel.")
122     print("=" * 72)
123
124     return w1, w2, b, t, sucesso
125
126
127 #
-----
128 # 4. Geracao do grafico para a funcao escolhida
129 #
-----
130 def gera_grafico(nome, w1, w2, b, t, sucesso):
131     os.makedirs(PASTA_SAIDA, exist_ok=True) # evita o
       FileNotFoundError
132
133     fig, ax = plt.subplots(figsize=(5.5, 5.5))
134
135     cores = ["#d62728" if v == -1 else "#1f77b4" for v in t]
136     ax.scatter(X[:, 0], X[:, 1], c=cores, s=220, edgecolor="k",
               zorder=3)
137
138     for (x1, x2), v in zip(X, t):
139         ax.annotate(f"({x1:+.0f},{x2:+.0f})  t={int(v):+d}", (x1
               , x2),
               textcoords="offset points", xytext=(0, 14),
               ha="center", fontsize=9)
140
141     xx = np.linspace(-1.8, 1.8, 50)
142
143     if sucesso:
144         if abs(w2) > 1e-9:
145             yy = -(w1 * xx + b) / w2
146             ax.plot(xx, yy, "k-", linewidth=2,
147                   label=f"$w_1$={w1:+.0f}, $w_2$={w2:+.0f},
148                         $b$={b:+.0f}")
149         elif abs(w1) > 1e-9:
150             xv = -b / w1
151             ax.axvline(xv, color="k", linewidth=2,
152                   label=f"$w_1$={w1:+.0f}, $w_2$={w2:+.0f},
153                         $b$={b:+.0f}")
154         ax.legend(loc="lower right", fontsize=8)
155         titulo = f"{nome}: linearmente separavel\n(Hebb converge
               )"
156         cor_titulo = "black"
157     else:
158         ax.text(0, 0, "NAO SEPARAVEL\n(Hebb falha)", ha="center"
               , va="center",
               fontsize=11, color="#d62728", fontweight="bold")

```

```

159     titulo = f"{nome}: NAO linearmente separavel"
160     cor_titulo = "#d62728"
161
162     ax.set_xlim(-1.8, 1.8)
163     ax.set_ylim(-1.8, 1.8)
164     ax.set_xticks([-1, 1])
165     ax.set_yticks([-1, 1])
166     ax.axhline(0, color="gray", linewidth=0.5)
167     ax.axvline(0, color="gray", linewidth=0.5)
168     ax.set_xlabel("$x_1$")
169     ax.set_ylabel("$x_2$")
170     ax.set_title(titulo, fontsize=11, color=cor_titulo)
171
172     fig.tight_layout()
173
174     nome_arquivo = nome.split(" ")[0].replace("(", "").replace(")", "").lower()
175     caminho = os.path.join(PASTA_SAIDA, f"hebb_{nome_arquivo}.png")
176     fig.savefig(caminho, dpi=170)
177     plt.show()
178     print(f"\nGrafico salvo em: {caminho}")
179
180
181 #
182
183 -----
184
185 # 5. Loop principal
186 #
187 -----
188
189 def main():
190     while True:
191         mostra_menu()
192         escolha = input("Escolha uma funcao (numero): ").strip()
193
194         if escolha == "0":
195             print("Encerrando.")
196             break
197
198         if escolha not in FUNCOES:
199             print("Opcao invalida, tente novamente.")
200             continue
201
202         nome, alvo = FUNCOES[escolha]
203         w1, w2, b, t, sucesso = mostra_resultado(nome, alvo)
204         gera_grafico(nome, w1, w2, b, t, sucesso)
205
206         input("\nPressione ENTER para voltar ao menu...")

```

```
203  
204 if __name__ == "__main__":  
205     main()
```